

**МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ**  
**НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА**



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ**  
**КОМП'ЮТЕРНИХ СИСТЕМ**

**Лабораторна робота № 2**

**«Створення простої комунікації**

**SW (client) ↔ UART ↔ HW (server)»**

**Виконав:**

**КІ-404**

**Рибка С.О.**

**Прийняв: Федак П.Р.**

**Львів - 2024**

## Завдання

1. Створіть просту комунікаційну схему SW (клієнт) ↔ UART ↔ HW (сервер).
  2. Клієнт повинен надсилати повідомлення на сервер, а сервер повинен змінювати повідомлення та повертати його клієнту.
  3. Створіть файл YML із такими функціями:
    - а. збірка всіх бінарних файлів (за потреби створити сценарії в папці сі);
    - б. запуск тестів;
    - в. створення артефактів із бінарними файлами та звітами про тестування;
- Згідно із моїм номером у списку маю таке завдання:

| Student number | Game            | Config format |
|----------------|-----------------|---------------|
| 17             | Tic-Tac-Toe 3x3 | JSON          |

## Теорія

UART (універсальний асинхронний приймач-передавач) — це периферійний пристрій для асинхронного послідовного зв'язку, у якому можна налаштувати формат даних і швидкість передачі. Він надсилає біти даних один за одним, від найменш значущого до найбільш значущого, обрамлені початковими та стоповими бітами. Рівні електричних сигналів обробляються зовнішньою по відношенню до UART схемою драйвера. Загальні рівні сигналу RS-232, RS-485 і необроблений TTL для коротких каналів налагодження.

Оскільки сучасні комп'ютери не мають апаратної підтримки вищезгаданих протоколів, то для зв'язку комп'ютера та мікроконтролера доводиться використовувати USB-UART "міст". Таким чином відбувається програмна емуляція UART.

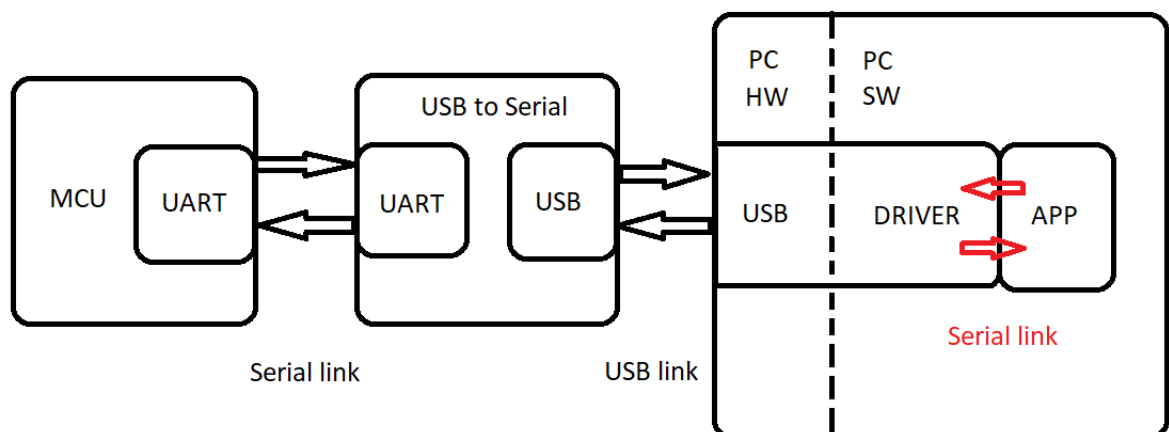


Рис. 1. Схема комунікації мікроконтролера та комп'ютера

## Виконання

Для виконання завдання я обрав відлагоджувальну плату на базі мікроконтролера ESP8266. Програму для мікроконтролера розробляв із використанням інтегрованого у Visual Studio Code середовища розробки PlatformIO (див рис. 3).

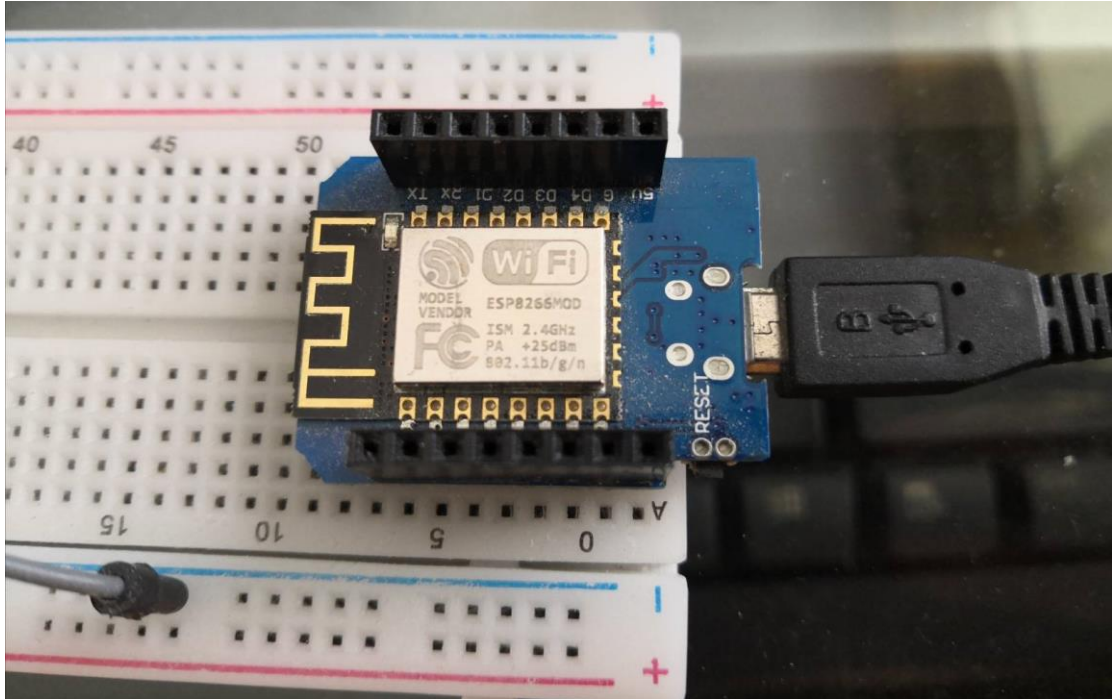


Рис. 2. Відлагоджувальна плата на базі ESP8266

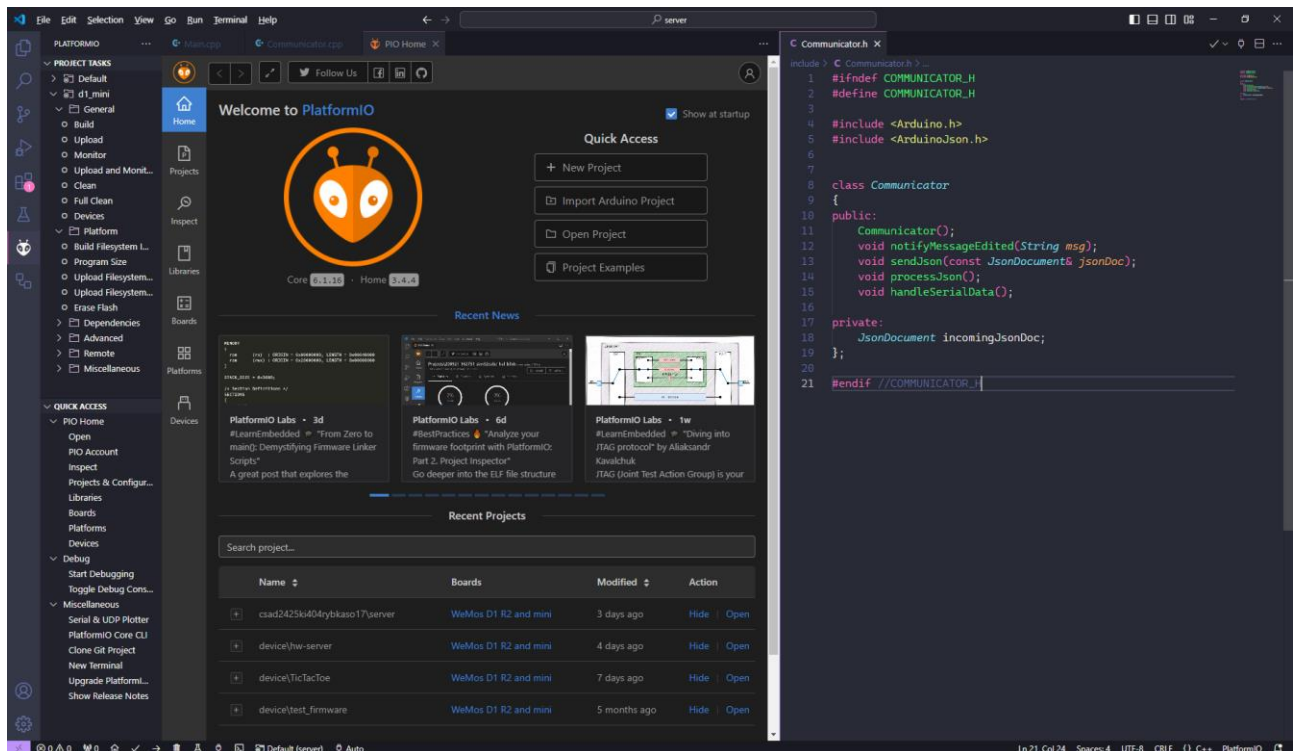


Рис. 3. Середовище розробки PlatformIO у Visual Studio Code

Для клієнтської десктопної програми я обрав мову програмування C++ та фреймворк Qt. На рис. 3. можна побачити скріншот розробленої програми.

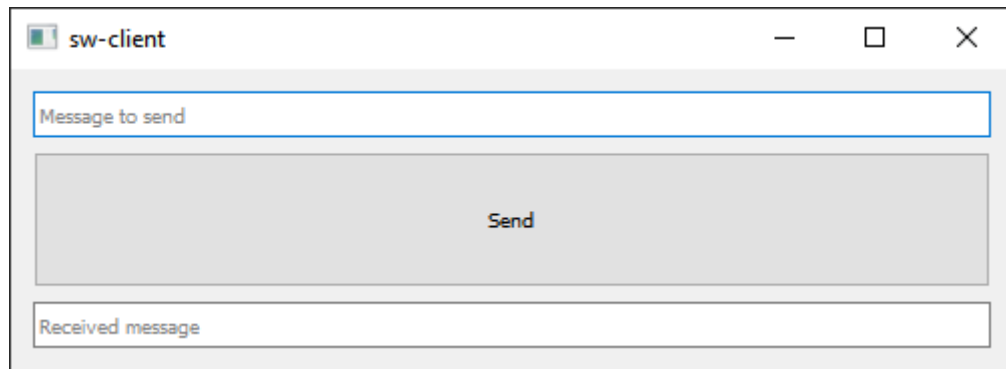


Рис. 4. Графічний інтерфейс розробленої програми

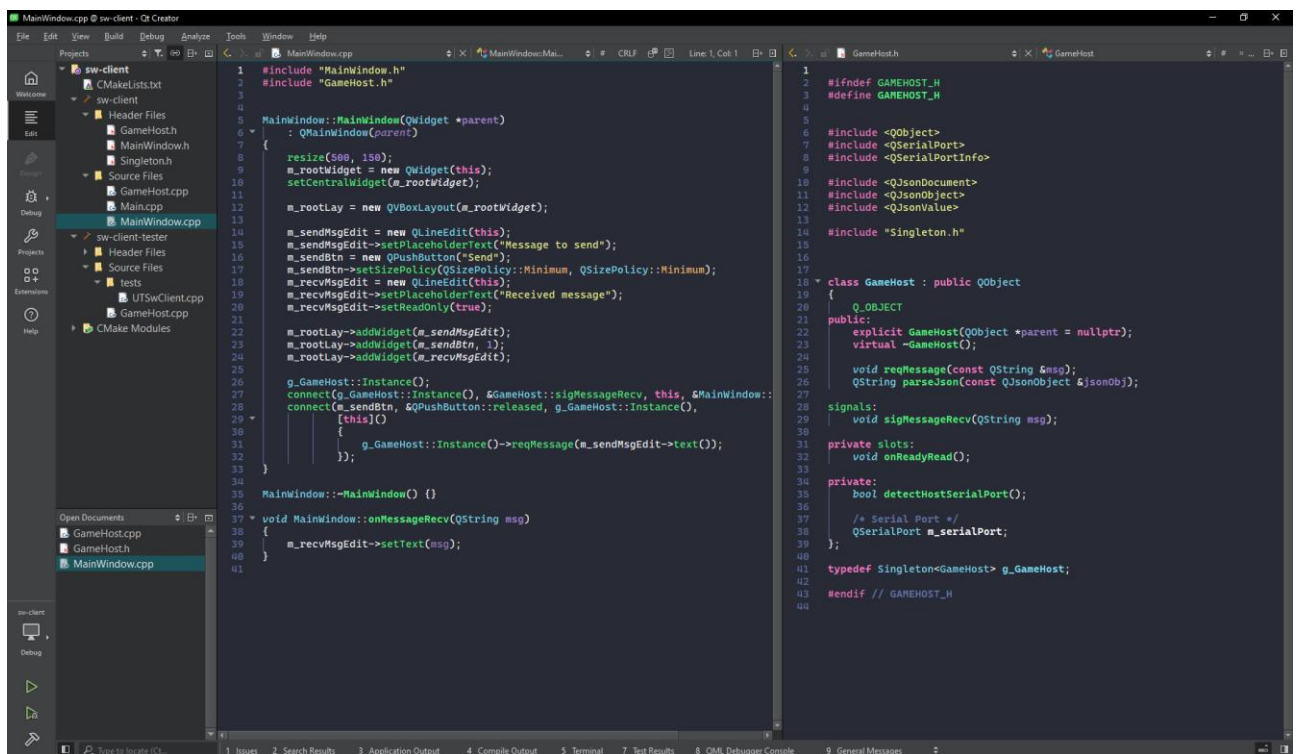


Рис. 5. Середовище розробки Qt

До клієнтської програми додав прості тести (див. рис. 7). На рис. 6. Зображено результати виконання тестів.

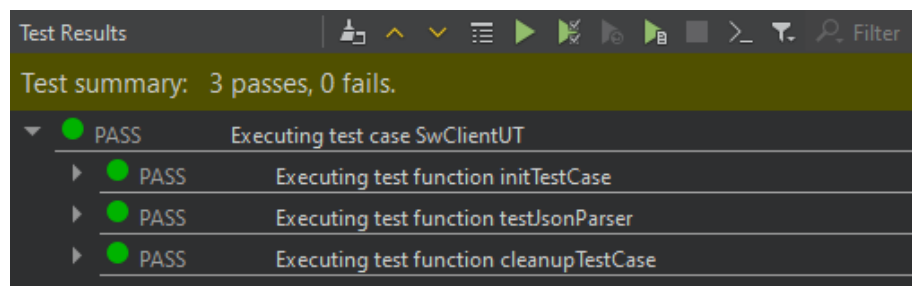


Рис. 6. Результат виконання тестів

```

1  #include <QTest>
2  #include "../GameHost.h"
3
4  class SwClientUT : public QObject
5  {
6      Q_OBJECT
7
8  private slots:
9      void initTestCase() {}
10
11      void cleanupTestCase() {}
12
13      void testJsonParser()
14      {
15          const QByteArray rawJson = QByteArray("{\"msg\":\"hello\"}");
16          const QJsonDocument jsonDoc = QJsonDocument::fromJson(rawJson);
17          QCOMPARE(g_GameHost::Instance()->parseJson(jsonDoc.object()), "hello");
18      }
19  };
20
21  QTEST_MAIN(SwClientUT)
22
23  #include "UTSwClient.moc"
24

```

Рис. 7. Клас для проведення тестування

Також додав до GitHub репозиторію файл ci.yml для автоматичної збірки проекту та запуску тестів.

```

.github > workflows > ci.yml
1  name: CI Qt CMake (Windows) and PlatformIO
2
3  on:
4      push:
5          branches:
6              - '*'
7      pull_request:
8          branches:
9              - '*'
10
11  env:
12      # Customize the CMake build type here (Release, Debug, RelWithDebInfo, etc.)
13      BUILD_TYPE: Debug
14
15  jobs:
16      sw-client-build:
17          runs-on: windows-latest
18
19          steps:
20              - uses: actions/checkout@v3
21
22              - name: Install Qt
23                # Installs the Qt SDK
24                uses: jurplel/install-qt-action@v2
25                with:
26                    version: '5.15.2'
27                    host: 'windows'
28                    target: 'desktop'
29                    arch: 'win64_msvc2019_64'
30
31              - name: Configure CMake
32                run: cmake ${github.workspace}/client -B ${github.workspace}/client/build -DCMAKE_BUILD_TYPE=${env.BUILD_TYPE}
33
34              - name: Build
35                # Build your program with the given configuration
36                run: cmake --build ${github.workspace}/client/build
37
38              - name: Upload Client build artifact
39                if: ${{ always() }}
40                uses: actions/upload-artifact@v4
41

```

Рис. 8. Файл ci.yml

GitHub успішно збіддив проєкт, провів тестування та вивантажив артефакти.

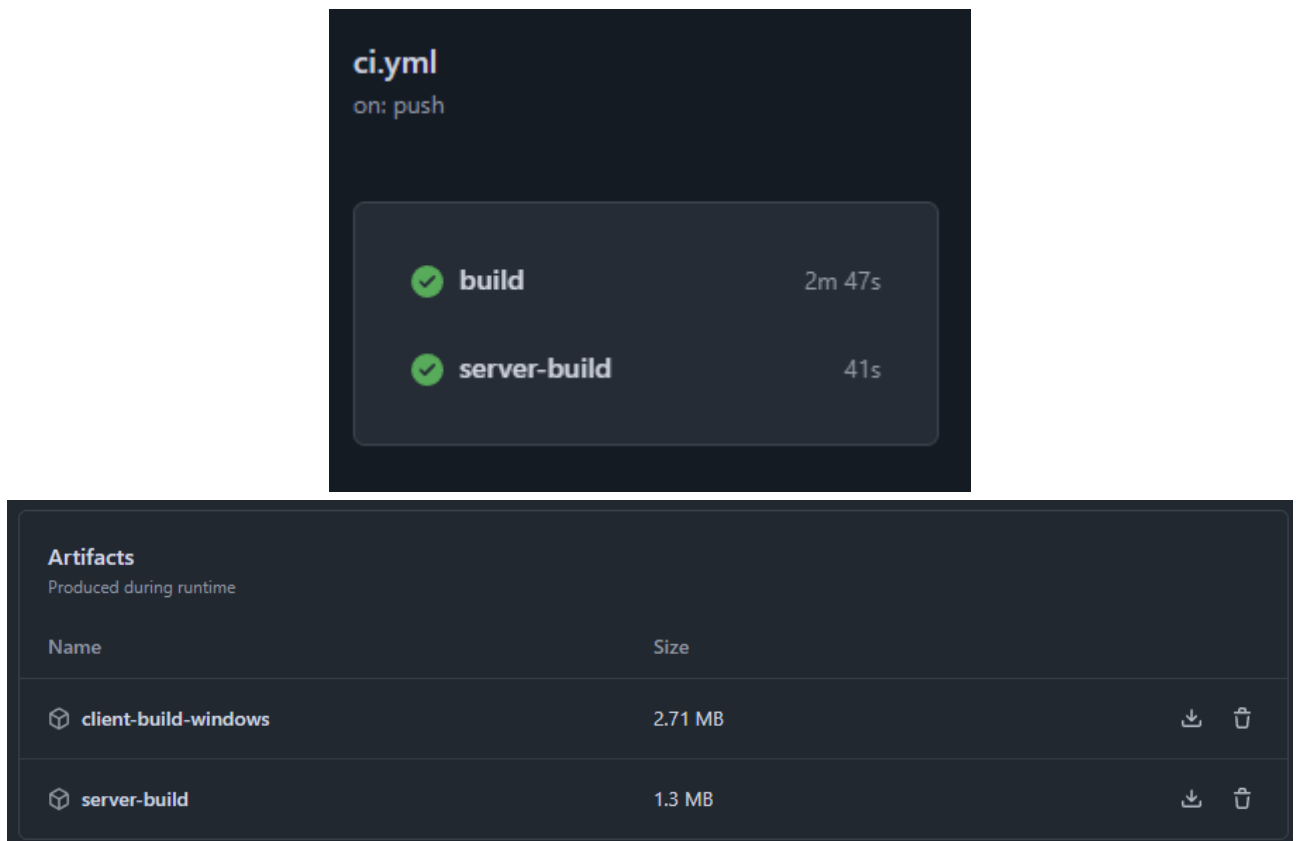


Рис. 9. Результати виконання `ci.yml`

## Висновок

Виконуючи лабораторну роботу, я створив комунікацію між десктопною програмою та мікроконтролером через віртуальном COM порт. Також налаштував `ci.yml` для роботи з GitHub Actions в якому прописав flow для білду проєктів та запуску тестів.